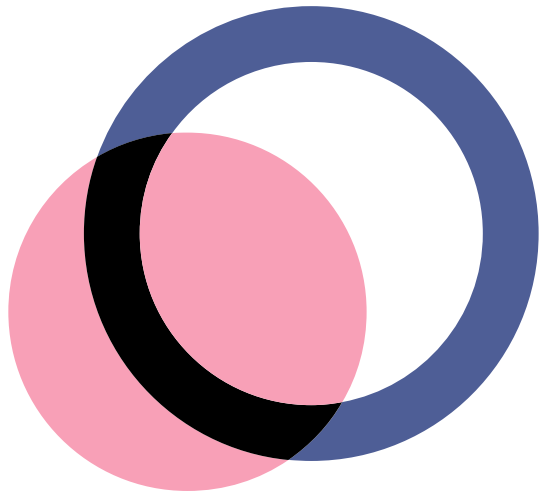
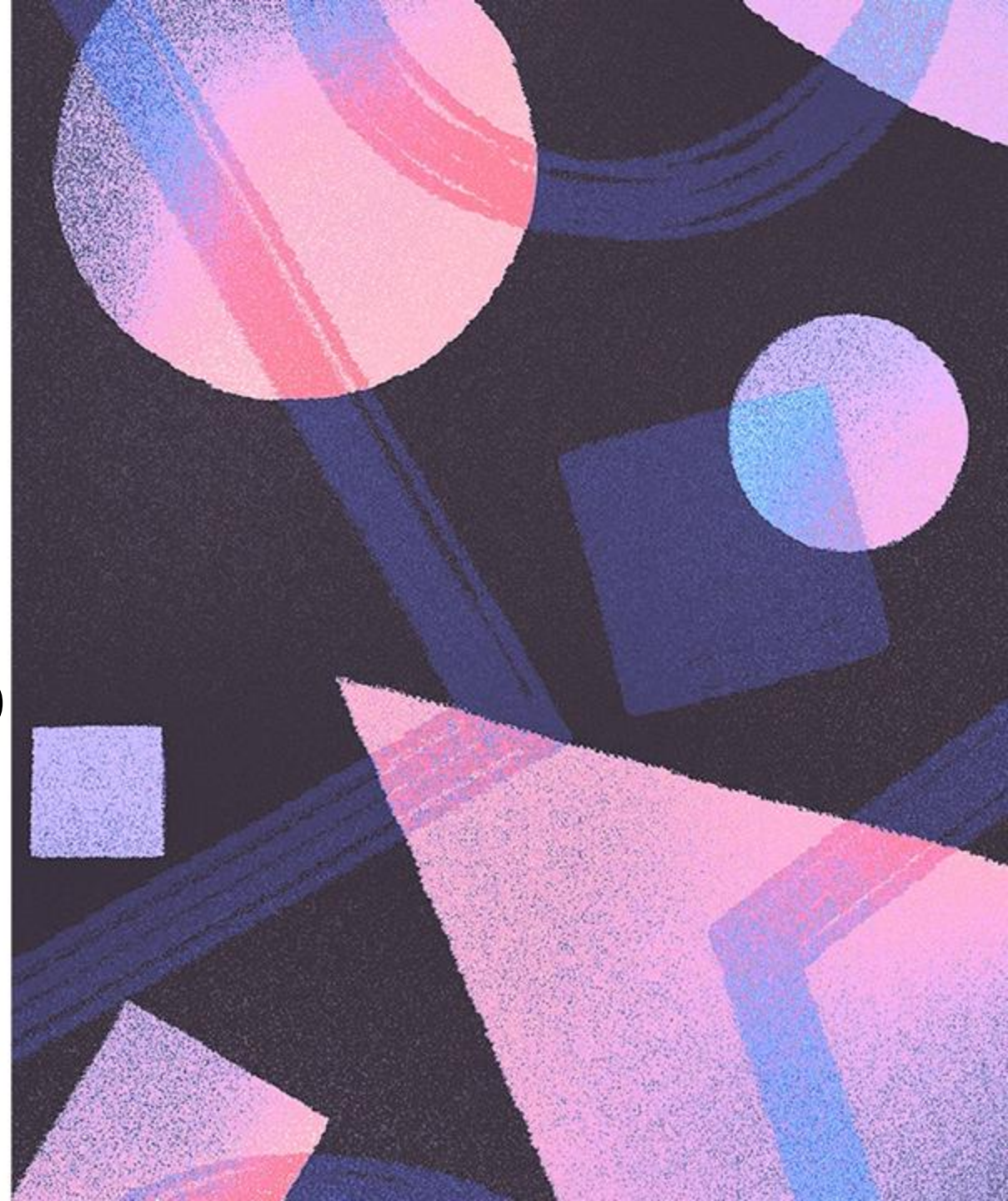




Design Patterns Lab

Lecture 9

> Adapter Pattern



Structural Design Patterns

- Structural Design Patterns (SDP) focus on system structure and relationships
- Concerned with:
 - connecting classes and objects
 - organizing dependencies
 - building larger software structures
- Goal:
 - flexible architecture
 - reusable systems
 - maintainable backend code

Why Structural Patterns Matter?

- Backend systems constantly integrate:
 - external APIs
 - payment gateways
 - cloud services
 - third-party SDKs
- Problem: every provider has different interfaces and formats
- Result without structure:
 - messy code
 - tight coupling
 - difficult maintenance
 - fragile integrations

Adapter Pattern

- Part of Structural Design Patterns
- Purpose: allow incompatible interfaces to work together
- Adapter acts as a translator between:
 - backend system
 - external service or SDK
- Converts one interface into another interface expected by the client

E-Commerce Scenario

- E-Commerce backend expects:
 - one standard payment interface
 - Existing third-party bank SDK uses different structure
- Problem:
 - incompatible method names
 - incompatible interfaces
- Solution:
 - place an Adapter between backend and SDK
 - Backend communicates only with standardized interface

Core Components

- Client: backend service using the payment system
- Target Interface: standard structure expected by backend
- Adaptee: external SDK or legacy service
- Adapter: converts calls between systems

Execution Flow

Steps:

1. Backend calls standard interface
2. Adapter receives request
3. Adapter translates request
4. External SDK executes operation
5. Result returns to backend transparently

Flow:

- Backend → Adapter → External SDK

Why Adapter Pattern is Powerful

- Decouples backend from external systems
- Centralizes compatibility logic
- Makes integrations cleaner and safer
- Allows replacing providers easily
- Protects internal architecture from external API changes

Real Examples:

- payment gateways
- SMS providers
- cloud storage APIs

Advantages & Disadvantages

Advantages:

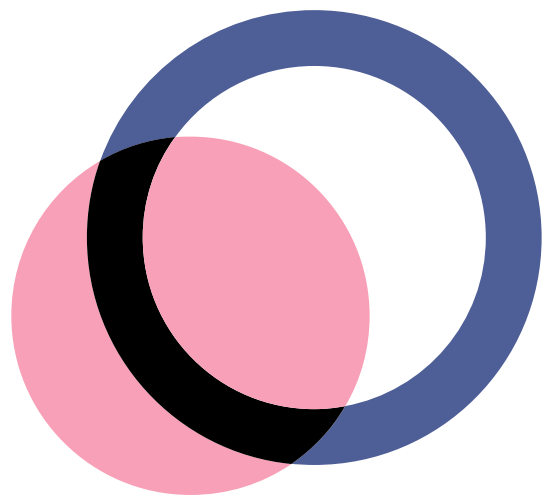
- Reusability
- Scalability
- Maintainability
- Flexibility
- Decoupling

Disadvantages:

- More classes
- Additional abstraction layers
- Slight increase in complexity

Activity: Multi Payment Gateway Integration System

- An E-Commerce backend integrates Stripe, Local Bank API, and Crypto Payment services
- Each provider exposes different method names and request structures
- The checkout service expects one unified PaymentProcessor interface
- Design adapters for each payment provider and connect them to the checkout system



**THANK
YOU**

